

INSO & MAIS



# Procesamiento de datos

Miguel Ángel Fernández Díaz  
2023-2024



# **Tema 3:**

## Spark Cluster

## 1. Gestores de recursos

- Standalone
- YARN
- Mesos
- K8S

## 2. Gestores de dependencias en Scala

- Maven
- Sbt

## 3. Despliegue de una aplicación en Spark

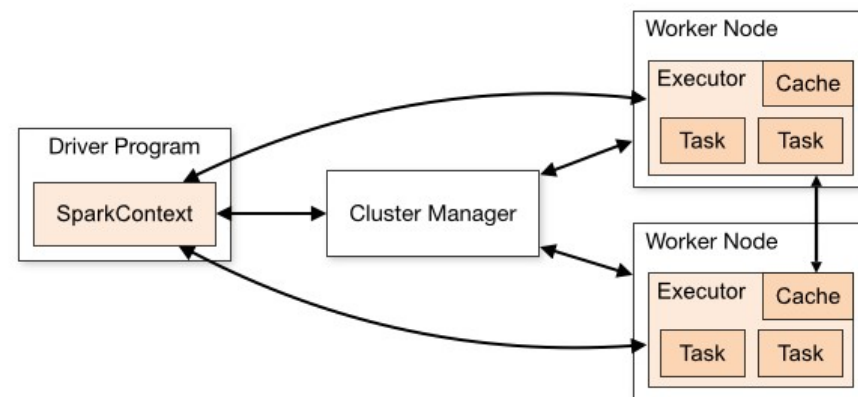
- Submit
- History Server
- Cancelación

## 4. Planificación de Jobs

- Pools
- Dynamic Allocation

# **1. Gestores de recursos**

- El Spark Driver coordina las tareas de la aplicación. Por ejemplo, una Spark-Shell
- Dentro del Spark Driver, lo primero que se crea es un Spark Context (conexión a un clúster de Executors)
- Los Worker Nodes son contenedores del Gestor de recursos que reservan recursos (cores, memoria y disco) en las máquinas en las que el Gestor de recursos está desplegado
- Cuando se levanta un Spark Driver, se pide al Gestor de recursos que reserve una especificación de recursos
- El Gestor de recursos, si tiene recursos suficientes, levanta Executors de Spark en sus Worker Nodes. Si no tiene recursos suficientes, el Gestor de recursos envía “ofertas” al SparkContext para que decida si acepta esa oferta o prefiere seguir esperando (durante un tiempo limitado) a que el Gestor de recursos le envíe una “oferta” mejor o una “oferta” con todos los recursos que pedía.
- Un Executor es un proceso JVM que cacheará datos y ejecutará tareas que le envía el Spark Driver a través del Spark Context. Además, hay comunicación entre los Executors para intercambiar datos (Shuffle)
- El Spark Driver se encarga de crear Jobs, a partir de RDDs, en Stages compuesto de Tasks.
- Si un Executor se cae, el Gestor de Recursos levanta un nuevo Executor y el Spark Driver se encarga de enviarle la tarea que falló.
- Esta arquitectura permite aislar el procesamiento entre las diferentes aplicaciones.
- Dado que el Spark Driver coordina los procesos entre los Executors, se aconseja que esté en la misma red que estos.



- Gestor de recursos incluido en la distribución Open-Source de Apache Spark
- NO es aconsejable para entornos productivos
- Fácil de configurar en un único nodo
- NO compatible con contenedores Docker
- Aislamiento de recursos
- Especificación de cores y memoria para el master y los workers
- Modo client vs Modo cluster
  - Client: El driver se levanta en la misma máquina en la que se ejecuta el cliente
  - Cluster: El driver se ejecuta dentro de un Worker del Gestor de recursos
    - Flag supervise: reinicia la aplicación si el proceso no devuelve 0
- El Máster se puede configurar en HA utilizando ZooKeeper
- REST API para lanzar aplicaciones
- Parámetros de recursos al lanzar la aplicación:
  - Driver:
    - --driver-memory
    - --driver-cores (solo en modo cluster)
  - Executors:
    - --total-executor-cores
    - --executor-cores
    - ¿Cómo se define el número de Executors que se van a levantar?



- Yet Another Resource Negotiator (Ecosistema Hadoop)
- Usado en entornos productivos
- --master yarn
  - Configuración a través de la distribución de yarn-site.xml
- Compatible con contenedores Docker
- Particionamiento Dinámico
- Despliegue de containers (workers)
- Configurable con recursos de GPUs y FPGAs
- Uso de colas (queue) de prioridad de YARN
- Modo client vs Modo cluster
  - Client: El driver se levanta en la misma máquina en la que se ejecuta el cliente
  - Cluster: El driver se ejecuta dentro de un Worker del Gestor de recursos
- Parámetros de recursos al lanzar la aplicación:
  - Driver:
    - --driver-memory
    - --driver-cores (solo en modo cluster)
  - Executors:
    - --num-executors
    - --executor-cores
    - ¿Cómo se calcula el número de cores total del clúster?



- Spark nació como un proyecto para utilizar Mesos
- Usado en entornos productivos
- `--master mesos://<IP>:7077`
- Compatible con contenedores Docker
- Particionamiento dinámico: Uso recursos adicionales si están disponibles
- Despliegue de agentes (workers)
- Configurable con recursos de GPUs y FPGAs
- El Máster se puede configurar en HA utilizando ZooKeeper
- Modo client vs Modo cluster
  - Client: El driver se levanta en la misma máquina en la que se ejecuta el cliente
  - Cluster: El driver se ejecuta dentro de un Worker del Gestor de recursos (modo supervise)
- Parámetros de recursos al lanzar la aplicación:
  - Driver:
    - `--driver-memory`
    - `--driver-cores` (solo en modo cluster)
  - Executors:
    - `--total-executor-cores`
    - `--executor-cores`
    - ¿Cómo se define el número de Executors que se van a levantar?



Apache  
**MESOS**™

- Proyecto desarrollado originalmente por Google
- Orquestación de contenedores
- Usado en entornos productivos
- `--master k8s://https://<k8s-apiserver-host>:<k8s-apiserver-port>`
- Contenedores Docker como uso principal
- Balanceo de carga y escalabilidad automáticos
- Despliegue de pods (workers)
- Configurable con recursos de GPUs y FPGAs
- Uso de volúmenes para datos shuffle
- Modo client vs Modo cluster
  - Client: El driver se levanta en la misma máquina en la que se ejecuta el cliente
  - Cluster: El driver se ejecuta dentro de un Worker del Gestor de recursos (modo supervise)
- Parámetros de recursos al lanzar la aplicación:
  - Driver:
    - `--spark.kubernetes.driver.request.cores`
    - `--spark.kubernetes.driver.limit.cores`
  - Executors:
    - `--spark.kubernetes.executor.request.cores`
    - `--spark.kubernetes.executor.limit.cores`

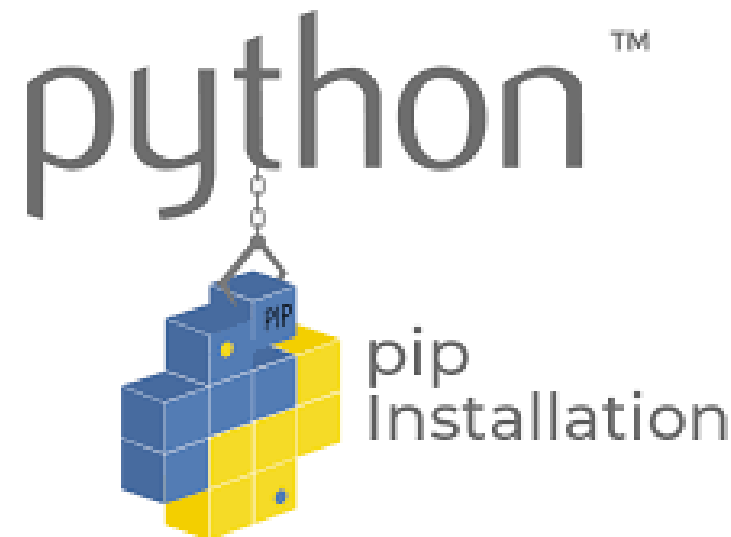


# kubernetes

## **2. Gestores de dependencias en Scala**

## ¿Qué es un gestor de dependencias?

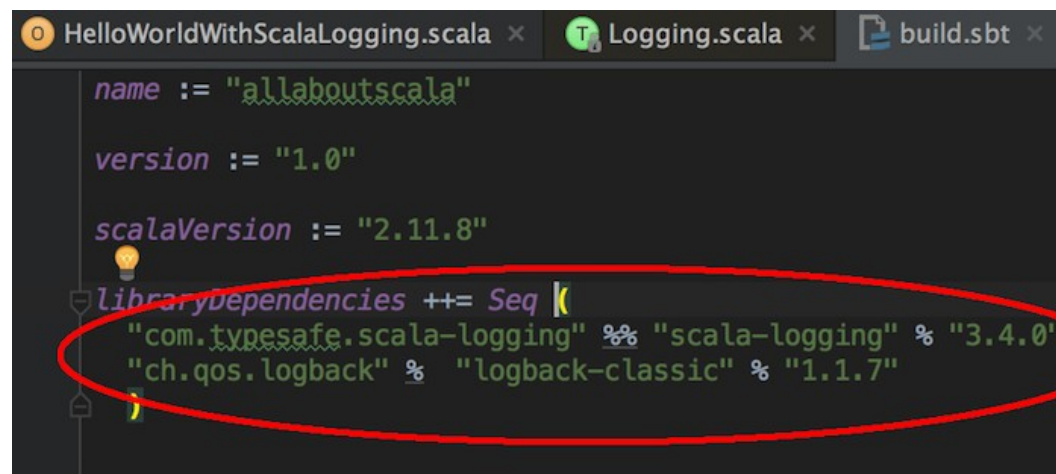
- Herramienta para añadir, priorizar y configurar los artefactos que componen un proyecto de programación
- Los artefactos pueden ser, por ejemplo, librerías, ficheros de configuración, paquetes, módulos
- Automatiza la instalación, actualización y gestión de las dependencias de un proyecto
  - Simplifica la gestión del ciclo de vida del Software para su desarrollo y distribución
- Puede ser específico para un lenguaje o de propósito general
- Tareas principales:
  - Instalación de dependencias: descarga e instalación
  - Resolución de dependencias: asegura compatibilidad entre versiones
  - Actualización de dependencias: simplifica el upgrade de versiones de librerías
  - Gestión de versiones: reproducibilidad del entorno
  - Aislamiento de Entornos: aislamiento de cada proyecto
- Ejemplos:
  - npm para JavaScript
  - pip para Python
  - Composer para PHP
  - Bundler para Ruby
  - Maven para Java
  - Sbt para Scala
  - Gradle para Android



- Gestión de proyectos y construcción de software desarrollado, principalmente, en Java.
- Simplifica y automatiza la construcción de artefactos, el manejo de dependencias y la distribución de proyectos.
- La definición de un proyecto se especifica en el fichero pom.xml, el cual tiene las siguientes secciones:
  - <project>: definición de groupId, artifactId y version
  - <properties>: definición de constantes como, por ejemplo, versiones
  - <dependencies>: listado de librerías, de las que depende el proyecto, definidas por groupId, artifactId y version
  - <build>: especificaciones de compilación, empaquetado, ejecución de tests, etc...
  - <plugins>: extensiones que contienen utilidades, especialmente, para procesos ejecutados en el build
  - <repositories>: definición de otros repositorios de artefactos y librerías
  - <profiles>: define configuraciones de acuerdo a diferentes perfiles
- Comandos más comunes:
  - Clean: elimina carpeta target
  - Compile: compila código sin generar un JAR. Genera archivos de clases en target
  - Test: ejecuta pruebas definidas en el proyecto
  - Package: compila, ejecuta pruebas y genera un JAR con los archivos binarios del proyecto
  - Install: Package + instalación del proyecto en el repositorio local
  - Deploy: Como install pero instala los artefactos del proyecto en un repositorio remoto
- Estos comandos se pueden combinar. Por ejemplo: mvn clean install
- Repositorios de librerías más populares:
  - Maven central: <https://central.sonatype.com/>
  - Maven repository: <https://mvnrepository.com/>

```
</plugins>
<finalName>MavenEnterpriseApp-ear</finalName>
</build>
<dependencies>
  <dependency>
    <groupId>com.mycompany</groupId>
    <artifactId>MavenEnterpriseApp-ejb</artifactId>
    <version>1.0-SNAPSHOT</version>
    <type>ejb</type>
  </dependency>
  <dependency>
    <groupId>com.mycompany</groupId>
    <artifactId>MavenEnterpriseApp-web</artifactId>
    <version>1.0-SNAPSHOT</version>
    <type>war</type>
  </dependency>
</dependencies>
</project>
```

- Simple Build Tool para proyectos del ecosistema Scala
- Permite, entre otras acciones, compilar, empaquetar, probar y gestionar dependencias
- La propia definición del proyecto se realiza mediante código Scala
- Construcciones incrementales: solo se actualizan aquellos artefactos a los que implique los últimos cambios
- Principales ficheros:
  - Build.sbt → definición de dependencias, configuración del compilador, tareas ad-hoc
  - project/build.properties → definición de la versión de SBT
  - project/plugins.sbt → extensiones para añadir funcionalidades e integrar otras herramientas
- Principales comandos:
  - Compile: compilación (incremental) del código fuente
  - Test: pruebas unitarias
  - Run: ejecución de la clase principal sin necesidad de empaquetar
  - Clean: elimina clases compiladas y artefactos JAR
  - Package: compilación + empaquetado de binarios en un JAR
  - Publish: publicar artefactos en un repositorio remoto
  - Reload: recarga la configuración de build.sbt para aplicar cambios sin reiniciar SBT



```
name := "allaboutscala"

version := "1.0"

scalaVersion := "2.11.8"

libraryDependencies ++= Seq(
  "com.typesafe.scala-logging" %% "scala-logging" % "3.4.0",
  "ch.qos.logback" % "logback-classic" % "1.1.7"
```

# **3. Despliegue de una aplicación**

## Maneras de desplegar una aplicación Spark

- Definir, desarrollar y empaquetar tu proyecto en fichero JAR:
  - El código binario debe contener un Main que levante al menos una SparkSession y NO se especifique el master
  - Spark-submit
  - Spark Standalone API REST
    - `http://[host:port]/[version]/submissions/[action]` → `http://localhost:6066/v1/submissions/create`
- Spark-shell
  - Interfaz interactiva para código incremental
- Ejecutar MainClass de un proyecto Java/Scala
  - Código de creación de SparkSession especificando Spark Master

```
import org.apache.spark.sql.SparkSession

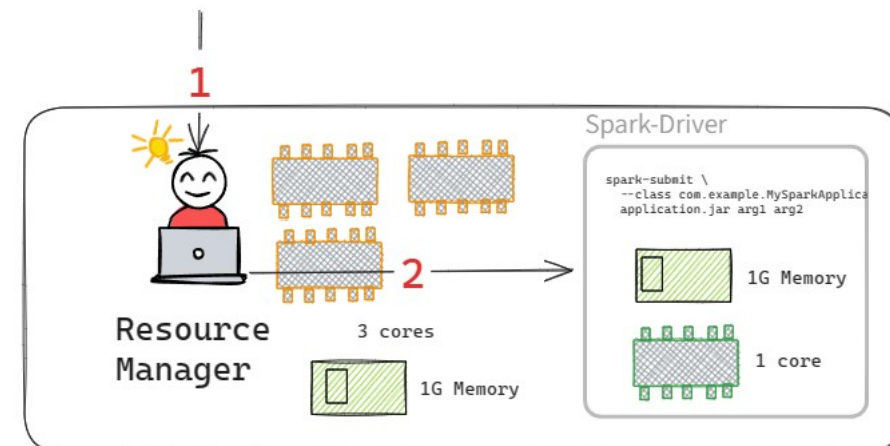
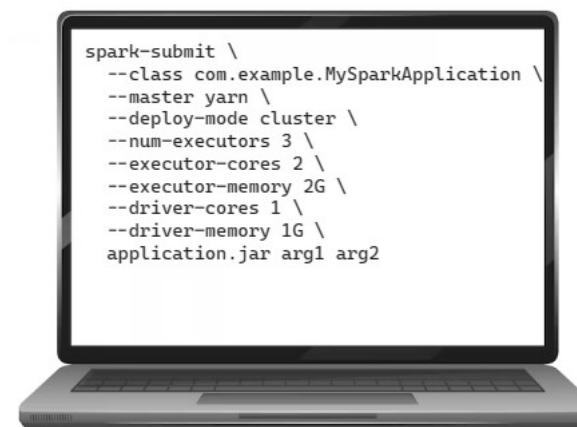
object Test extends App {

  val spark = SparkSession.builder()
    .master("local[*]")
    .appName("SQL Queries tester")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.execution.datasources.v2.jdbc.JDBCTableCatalog")
    .config("spark.sql.catalog.spark_catalog.url", "jdbc:postgresql://localhost:5432/postgres?user=postgres&password=postgres")
    //.config("spark.sql.catalog.spark_catalog.dbtable", "test")
    .getOrCreate()
```

```
curl -X POST http://<tu-spark-master>:6066/v1/submissions/create \
--header "Content-Type:application/json;charset=UTF-8" \
--data '{
  "action": "CreateSubmissionRequest",
  "appArgs": ["arg1", "arg2"],
  "appResource": "hdfs://localhost:9000/apps/apps/hello_world.jar",
  "clientSparkVersion": "3.0.1",
  "environmentVariables": {
    "SPARK_ENV_LOADED": "1"
  },
  "mainClass": "mi.package.MiApp",
  "sparkProperties": {
    "spark.driver.memory": "1g",
    "spark.executor.instances": "2",
    "spark.executor.memory": "1g",
    "spark.master": "spark://<tu-spark-master>:7077",
    "spark.app.name": "MiApp",
    "spark.submit.deployMode": "cluster"
  }
}'
```

# spark-submit

- Utilidad de Apache Spark para lanzar aplicaciones al clúster para su ejecución
- Admite proyectos escritos en Scala, Java, Python y R
- Es la manera más habitual de desplegar una aplicación de Spark
- Sintaxis básica:
  - `spark-submit --class <AppMainClass> --master <master> <jar-file>`
- Otras opciones:
  - `--deploy-mode`
  - `--name`
  - `--jars`
  - `--package` (coordenadas maven)
  - `--files`
  - `--conf`
  - `--driver-memory`
  - `--executor-memory`
- También permite obtener el estado de una aplicación:
  - `spark-submit --status [submission ID] --master <master>`



- La Web UI de Spark, cuya URL es `http://<driver-node>:4040`, es una aplicación que corre dentro del mismo proceso del Driver
  - Guarda estadísticas en memoria
    - Estas estadísticas no se pueden recuperar una vez que el Driver se detiene
- History Server es una funcionalidad que involucra 2 procesos:
  - Configurar el Spark Driver para que escriba las estadísticas también en HDFS
    - `spark.eventLog.enabled: true`
    - `spark.eventLog.dir: hdfs://namenode/shared/spark-logs`
  - Levantar la Web UI de History Server para mostrar información de aplicaciones finalizadas y ejecutándose
    - `sbin/start-history-server.sh`
    - `http://<server-url>:18080`
    - Propiedad `spark.history.fs.logDirectory: hdfs://namenode/shared/spark-logs`

## 3.0.0 History Server

Event log directory: `s3a://typeform-c`

Last updated: 2020-09-28 07:41:17

Client local time zone: Europe/Madrid

Search:

| Version | App ID                                                 | App Name          | Started             | Completed           | Duration | Spark User | Last Updated        | Event Log                |
|---------|--------------------------------------------------------|-------------------|---------------------|---------------------|----------|------------|---------------------|--------------------------|
| 3.0.0   | <a href="#">spark-178ea83302b84fe3b3a6aa4dcd6e5f01</a> | ValidationRunner  | 2020-09-27 16:31:16 | 2020-09-28 07:40:16 | 15.2 h   | root       | 2020-09-28 07:40:17 | <a href="#">Download</a> |
| 3.0.0   | <a href="#">spark-812a4728d8524a88b90ce8d3d738e461</a> | ValidationRunner  | 2020-09-26 22:10:59 | 2020-09-27 16:30:17 | 18.3 h   | root       | 2020-09-27 16:30:19 | <a href="#">Download</a> |
| 3.0.0   | <a href="#">local-1601164970245</a>                    | TransformerRunner | 2020-09-27 02:02:48 | 2020-09-27 02:03:15 | 27 s     | root       | 2020-09-27 02:03:16 | <a href="#">Download</a> |
| 3.0.0   | <a href="#">spark-7e7e631ab2ae458eb74c0eca1569f0cf</a> | ValidationRunner  | 2020-09-26 20:40:49 | 2020-09-26 22:10:04 | 1.5 h    | root       | 2020-09-26 22:10:05 | <a href="#">Download</a> |
| 3.0.0   | <a href="#">spark-9018fdd367614b11a556b5949dc3f129</a> | ValidationRunner  | 2020-09-23 16:33:54 | 2020-09-26 20:40:04 | 76.1 h   | root       | 2020-09-26 20:40:05 | <a href="#">Download</a> |
| 3.0.0   | <a href="#">local-1601078585262</a>                    | TransformerRunner | 2020-09-26 02:03:03 | 2020-09-26 02:03:28 | 25 s     | root       | 2020-09-26 02:03:29 | <a href="#">Download</a> |
| 3.0.0   | <a href="#">local-1600867597958</a>                    | TransformerRunner | 2020-09-23 15:26:36 | 2020-09-23 15:27:08 | 33 s     | root       | 2020-09-23 15:27:09 | <a href="#">Download</a> |

Showing 1 to 7 of 7 entries

[Show incomplete applications](#)

## ¿Cómo cancelar un Job o App de Spark?

### ▼ Active Jobs (1)

Page:  1 Pages. Jump to  . Show  items in a page.

| Job Id ▼ | Description                                                                | Submitted           | Duration | Stages: Succeeded/Total | Tasks (for all stages): Succeeded/Total |
|----------|----------------------------------------------------------------------------|---------------------|----------|-------------------------|-----------------------------------------|
| 7        | count at <console>:26<br><a href="#">show at &lt;console&gt;:26</a> (kill) | 2019/08/10 17:50:13 | 17 s     | 0/2                     | <div><div>0/5 (4 running)</div></div>   |

Page:  1 Pages. Jump to  . Show  items in a page.

### ▼ Completed Jobs (7)

Page:  1 Pages. Jump to  . Show  items in a page.

| Job Id ▼ | Description                                                                              | Submitted           | Duration | Stages: Succeeded/Total | Tasks (for all stages): Succeeded/Total |
|----------|------------------------------------------------------------------------------------------|---------------------|----------|-------------------------|-----------------------------------------|
| 6        | <a href="#">show at &lt;console&gt;:26</a><br><a href="#">show at &lt;console&gt;:26</a> | 2019/08/10 17:49:30 | 0.4 s    | 1/1                     | <div><div>1/1</div></div>               |
| 5        | <a href="#">show at &lt;console&gt;:28</a><br><a href="#">show at &lt;console&gt;:28</a> | 2019/08/10 17:48:32 | 0.8 s    | 3/3                     | <div><div>9/9</div></div>               |
| 4        | <a href="#">show at &lt;console&gt;:28</a><br><a href="#">show at &lt;console&gt;:28</a> | 2019/08/10 17:47:40 | 2 s      | 3/3                     | <div><div>9/9</div></div>               |

- Spark Driver UI
  - En la descripción del Job, aparece la opción de Kill
- Spark Master UI (cluster mode)
  - Botón kill al lado de Submission ID
- Spark-submit permite parar una aplicación
  - `spark-submit --kill [submission ID] --master [spark://...]`
- Programáticamente se puede cancelar un consulta mediante el `setJobGroup` y el `cancelJobGroup` del objeto `SparkContext`

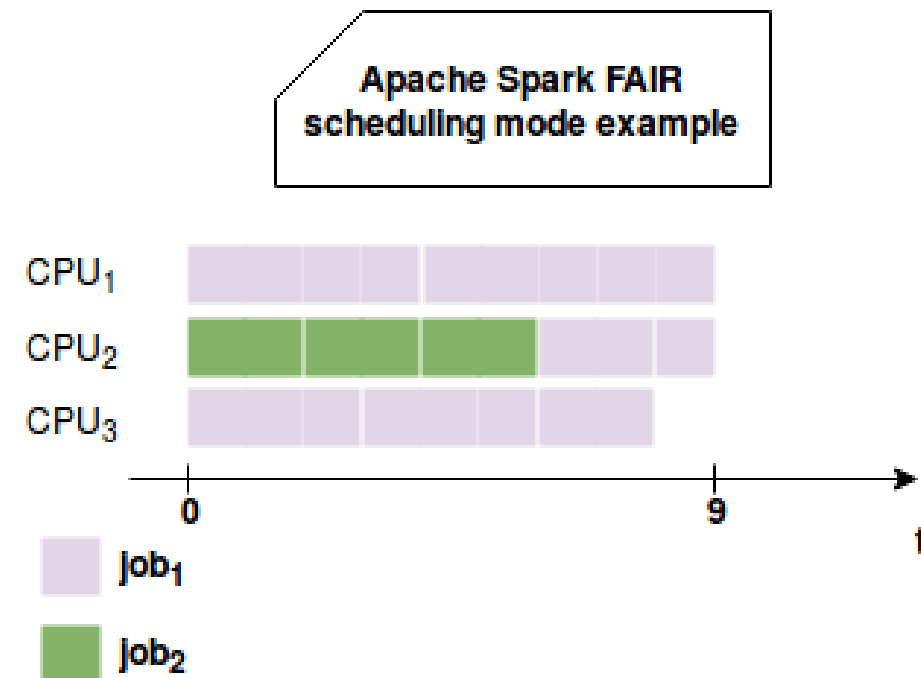
```
20/01/01 12:34:56 INFO SparkContext: Submitted application: mi_aplicacion
20/01/01 12:34:57 INFO Client: Application report for mi_aplicacion (state: ACCEPTED)
20/01/01 12:34:58 INFO Client: Client created submission ID: my_submission_id
```

```
// In the main thread:
sc.setJobGroup("some_job_to_cancel", "some job description")
sc.parallelize(1 to 10000, 2).map { i => Thread.sleep(10); i }.count()
```

```
// In a separate thread:
sc.cancelJobGroup("some_job_to_cancel")
```

# **4. Planificación de Jobs**

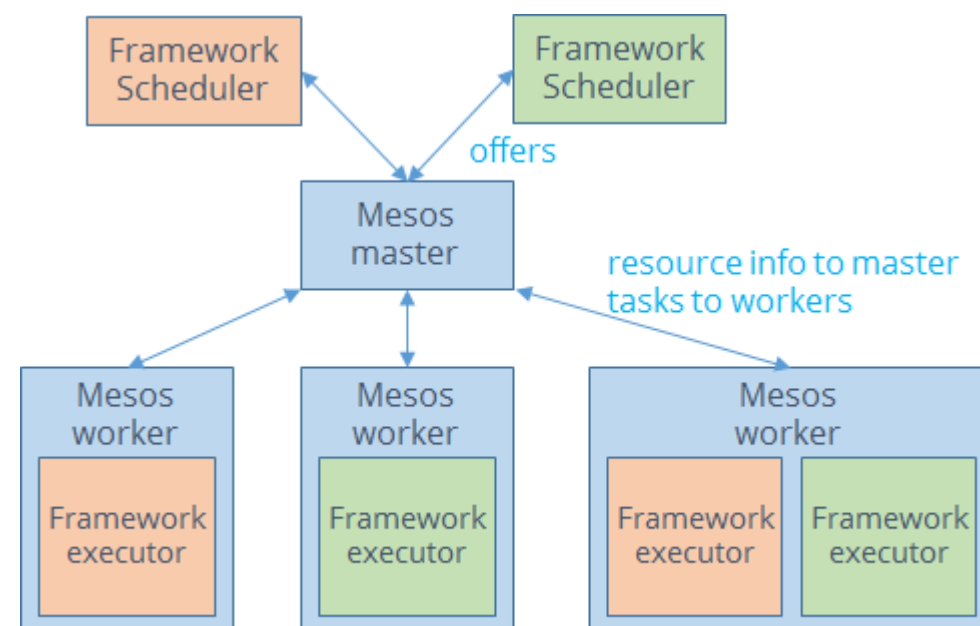
- Escenario multiusuario
- Desde una Aplicación de Spark (instancia de SparkContext), se puede ejecutar varias consultas en paralelo
- Un hilo de ejecución por usuario o consulta
- Por defecto, el planificador de Spark se comporta como una cola FIFO
  - La consulta que está a la cabeza de la cola tiene prioridad sobre todos los recursos disponibles
  - Si la consulta prioritaria realiza tareas que no requiere el uso de todo el clúster, entonces otras consultas pueden utilizar los recursos sobrantes
  - Si la consulta prioritaria es pesada, otras consultas ligeras se retrasan significativamente
- Es posible configurar un planificador “Fair” (Equitativo)
  - Reparto de recursos “Round Robin” (Por turnos)
  - Consultas ligeras avanzan y se ejecutan sin tener que esperar a la finalización de consultas pesadas
  - Compatible con Standalone, YARN, Mesos y K8s



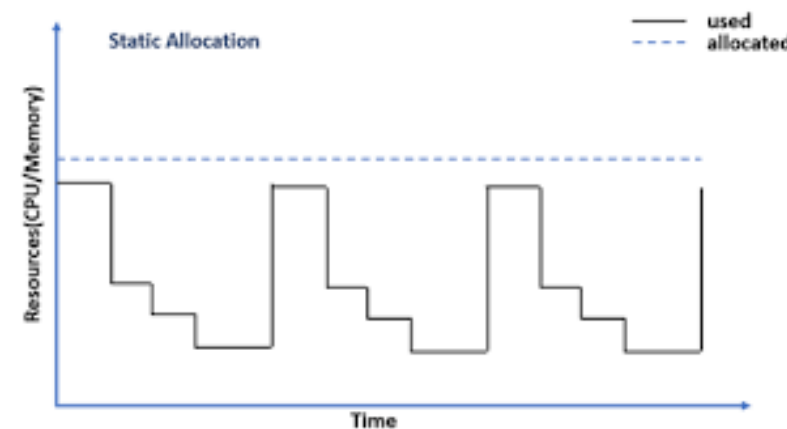
- El planificador Fair permite agrupar consultas en pools
- Permite crear colas de prioridad
- Permite crear colas por usuario para repartir los recursos equitativamente entre usuarios, independientemente de las consultas concurrentes de cada usuario
- Modelo basado en el planificador Fair de Hadoop
- Sin tocar la configuración, todas las consultas de todos los hilos van a pool por defecto
- Se puede definir un pool por hilo con:
  - `sc.setLocalProperty("spark.scheduler.pool", "pool1")`
- Para volver al pool por defecto:
  - `sc.setLocalProperty("spark.scheduler.pool", null)`
- Por defecto, todos los pools obtienen los mismos recursos pero dentro de cada pool, las consultas se ejecutan como una cola FIFO
- Configuración de pools:
  - Propiedad `spark.scheduler.allocation.file` o fichero `fairscheduler.xml` en classpath
    - `schedulingMode`: FIFO o FAIR
    - `weight`: mayor peso = mayor prioridad
    - `minShare`: mínimo de cores que el planificador debe asignar a un pool

```
<?xml version="1.0"?>
<allocations>
  <pool name="production">
    <schedulingMode>FAIR</schedulingMode>
    <weight>1</weight>
    <minShare>2</minShare>
  </pool>
  <pool name="test">
    <schedulingMode>FIFO</schedulingMode>
    <weight>2</weight>
    <minShare>3</minShare>
  </pool>
</allocations>
```

- Cada aplicación de Spark obtiene un conjunto independiente de ejecutores (procesos JVM) que ejecuta tareas y almacena datos
- En entornos compartidos, la configuración de recursos de cada aplicación afecta al funcionamiento global del entorno
- La manera más simple de repartir los recursos de un entorno es mediante el particionamiento estático.
  - Cada aplicación bloquea una cantidad máxima de recursos
    - Standalone: `spark.cores.max`
    - Mesos: `spark.cores.max`
    - YARN: `spark.executor.instances + spark.executor.cores`
- Para Mesos, existe una segunda opción que consiste en utilizar una compartición dinámica de Cores en Mesos
  - Se bloquea el uso memoria por aplicación (`spark.executor.memory`), pero si no hay tareas ejecutándose en una aplicación, entonces Mesos puede prestar cores a otras aplicaciones.
  - Sin embargo, si esa misma aplicación empieza a ejecutar tareas, sufrirá una latencia en recuperar el control de esos cores



- Mecanismo para ajustar dinamicamente los recursos ocupados de cada aplicación en función de la carga de trabajo
- Opción desactivada por defecto en Standalone, YARN, Mesos y K8s
- Las aplicaciones devuelven recursos al gestor cuando no los están utilizando y los solicitan de nuevo cuando se requieren
  - Se paran y se levantan executors
- Para su uso, se necesita que los executors no borren sus ficheros de shuffle
- 2 Opciones para activar Dynamic Allocation:
  - `spark.dynamicAllocation.enabled + spark.dynamicAllocation.shuffleTracking.enabled`
  - `spark.dynamicAllocation.enabled + spark.shuffle.service.enabled + Servicio de Shuffle externo`
    - Ejemplo: `start-mesos-shuffle-service.sh` en todos los Mesos Workers
- Parámetros para la adquisición de recursos
  - `spark.dynamicAllocation.schedulerBacklogTimeout: 1s`
  - `spark.dynamicAllocation.sustainedSchedulerBacklogTimeout`
    - Se añaden executors en cada ronda incrementando la potencia de 2: 2, 4, 8, ...
- Parámetros para la liberación de recursos
  - `spark.dynamicAllocation.executorIdleTimeout: 60s`
  - `spark.dynamicAllocation.cachedExecutorIdleTimeout: infinito`
- Otros parámetros:
  - `spark.dynamicAllocation.maxExecutors: infinito`
  - `spark.dynamicAllocation.minExecutors: 0`
  - `spark.dynamicAllocation.initialExecutors`





 Calle Playa de Liencres, 2 bis  
(entrada por calle Rozabella)  
Parque Europa Empresarial  
Edificio Madrid  
28290 Las Rozas, Madrid

 900 373 379  [info@u-tad.com](mailto:info@u-tad.com)

 [SOLICITA MÁS INFORMACIÓN](#)



CENTRO ADSCRITO A:



PROYECTO COFINANCIADO POR:

